SILENTVECTOR

Distributed Mini Operating System

Powered by PHANTOM OS

"The master node silently monitors all nodes — always watching, never seen."

Contents

1. Project Overview 3
   1.1 Problem Statement 3
   1.2 Why "Silent Vector"? 3
2. System Architecture 4
   2.1 Folder Structure 4
   2.2 Communication Protocol 4
3. Worker Node — Startup & Registration 5
4. Phantom Shell — Command Interface 5
   4.1 Command Reference 6
5. Feature Demonstrations 7
   5.1 Node Ping — Connectivity Check 7
   5.2 System Information — sysinfo 7
   5.3 Task Management — tasks 8
6. Network Topology 9
7. Fault Tolerance & Recovery 9
   7.1 Scenario A — Node Online (No Recovery Needed) 9
   7.2 Scenario B — Node Offline (Recovery Triggered) 10
8. Live Dashboard 11
9. Audit Logging — logs 11
10. Cinematic Demo — auto_demo & replay 12
   10.1 Auto Demo — Multi-Step Walkthrough 12
   10.2 Replay Mode 13
11. Setup & Deployment Guide 14
   11.1 Prerequisites 14
   11.2 Step 1 — Start Worker Node(s) 14
   11.3 Step 2 — Launch Phantom Shell 14
   11.4 Step 3 — Configure Nodes 14
12. Summary & Conclusion 15

1. Project Overview

Silent Vector is a Python-based distributed mini operating system simulation built to demonstrate a master shell controlling multiple worker nodes across a network. The system was designed with a focus on real-time visibility, task orchestration, and fault resilience.

The name "Silent Vector" reflects the core philosophy of the architecture: the master node silently monitors all worker nodes — collecting telemetry, managing tasks, detecting failures, and directing recovery — without interrupting operations. It is the invisible force that keeps the distributed system coherent.

1.1 Problem Statement

Traditional OS education often lacks hands-on, interactive experiences for distributed task management and node monitoring. Silent Vector provides a visual, real-time, and interactive environment where users can:

- Simulate node communication across a network
- Observe task migration between nodes in real time
- Test fault tolerance mechanisms and auto-recovery simulation
- Monitor live CPU and RAM usage via ASCII dashboards
- Study protocol-level communication through a custom packet layer

1.2 Why "Silent Vector"?

The master node silently monitors all worker nodes — always observing system health, tracking tasks, detecting failures, and triggering recovery actions — without any visible footprint on the worker side. Like a vector in mathematics, it has direction and magnitude: silent in nature but powerful in effect

2. System Architecture

Silent Vector follows a Master-Slave distributed architecture. The master shell (Phantom Shell) acts as the central control plane, communicating with one or more worker nodes over TCP sockets. Each worker node registers itself with a unique Node ID and reports system metrics on demand.

2.1 Folder Structure

File / Folder	Purpose
master/phantom_shell.py	Main interactive shell — entry point for all commands
master/node_manager.py	Manages node registry, broadcasting, and packet dispatch
master/dashboard.py	Renders the live ASCII dashboard for node monitoring
worker/worker_node.py	Worker node process — listens on TCP, responds to master
protocol/packet.py	Custom packet layer with headers and checksum validation
shared/config.json	Node topology configuration (IPs and ports)
shared/phantom.log	Audit log — records all events and commands
shared/replay_demo.json	Cinematic replay script for demo mode
sandbox/demo_file.txt	Sample file for sandbox transfer simulation

2.2 Communication Protocol

All inter-node communication is handled through a custom protocol layer implemented in packet.py. Each packet contains:

- Source and destination node identifiers
- Packet type: PING, SYSINFO, TASK_START, TASK_STOP, MIGRATE, etc.
- Payload data serialized as JSON
- Checksum for data integrity validation

3. Worker Node — Startup & Registration

Worker nodes are launched independently on any terminal or remote machine. Each node generates a unique Node ID, binds to a specified TCP port, and begins listening for incoming packets from the master shell.



Figure 1: Worker node startup — Node ID node-2613 listening on port 9001

In the screenshot above, the worker node successfully starts, assigns itself the identifier node-2613, and confirms it is actively listening on port 9001. This node registration is essential for the master to discover and communicate with the worker.

4. Phantom Shell — Command Interface

The Phantom Shell is the central command interface of Silent Vector. It provides a styled terminal UI with the PHANTOM branding, and exposes a full command set for interacting with the distributed system.



Figure 2: Phantom Shell — Main interface showing all available commands

4.1 Command Reference

Command	Description
ping	Ping all registered worker nodes and display response status
sysinfo	Query CPU usage, RAM usage, OS, and hostname from all nodes
tasks	List all currently running and waiting tasks on all nodes
start	Start a new task and distribute it to all connected nodes
stop	Stop a running task by providing its Task ID
migrate	Migrate a task from one node to another node
dashboard	Launch the live ASCII node monitoring dashboard
logs	Display the last N entries from the Phantom OS audit log
replay	Run the cinematic replay demo mode
auto_demo	Execute full automated cinematic distributed OS scenario
topology	Display the current network topology map of all nodes
fault_test	Detect offline nodes and simulate fault recovery actions
help	Show the command reference panel
exit	Exit the Phantom Shell gracefully

5. Feature Demonstrations

5.1 Node Ping — Connectivity Check

The ping command broadcasts a PING packet to all registered worker nodes and displays the response time and online status. This is the primary health-check mechanism used before executing any distributed operations.



Figure 3: ping — Node 192.168.100.128:9001 responds PONG with status ONLINE

5.2 System Information — sysinfo

The sysinfo command queries each worker node for its current system state. The response includes the Node ID, operating system, hostname, online status, and real-time CPU and RAM utilization represented as progress bars.

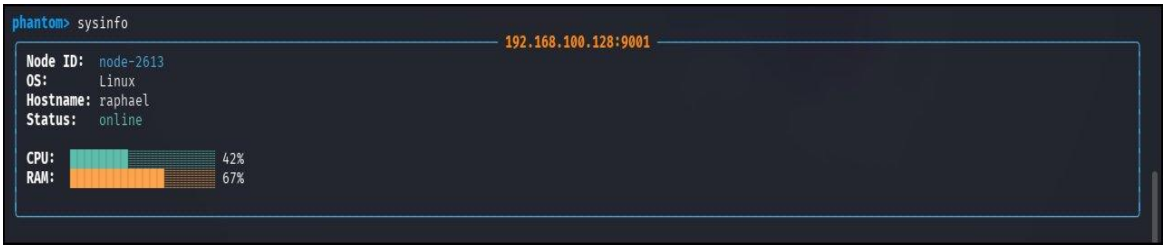


Figure 4: sysinfo — Node node-2613 reporting CPU: 42%, RAM: 67%

5.3 Task Management — tasks

The tasks command retrieves all active and queued tasks from every connected node. Each task entry shows its ID, name, current status (running / waiting), and completion progress percentage.

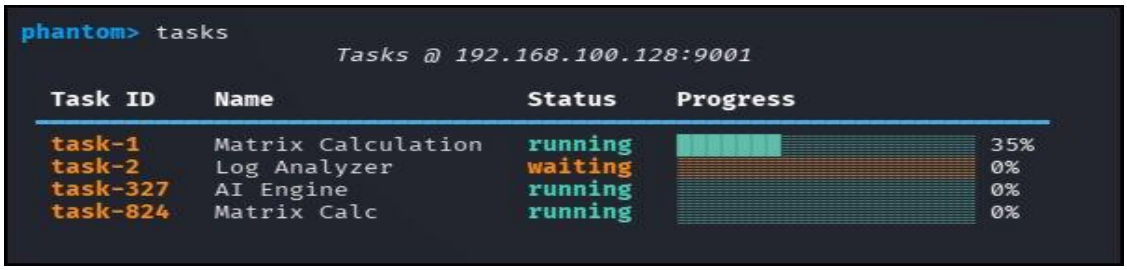


Figure 5: tasks — Active task list including Matrix Calculation, Log Analyzer, and AI Engine

6. Network Topology

The topology command renders a visual ASCII tree of the network topology. It shows the Master Node at the root and lists all connected worker nodes as leaf nodes, along with their address and current online/offline status.



Figure 6: topology — Master Node with worker 192.168.100.128:9001 showing ONLINE status

This view is essential for quickly verifying the network configuration and confirming which nodes are reachable before issuing distributed commands. The topology is dynamically loaded from shared/config.json, making it easy to add or remove nodes without code changes.

7. Fault Tolerance & Recovery

Fault tolerance is one of the core pillars of Silent Vector. The fault_test command actively probes each registered node and generates a Fault Detection Report. Based on availability, it prescribes a recovery action for each node.

7.1 Scenario A — Node Online (No Recovery Needed)

When the worker node is running and reachable, the fault_test reports the node as ONLINE and confirms no recovery action is required.

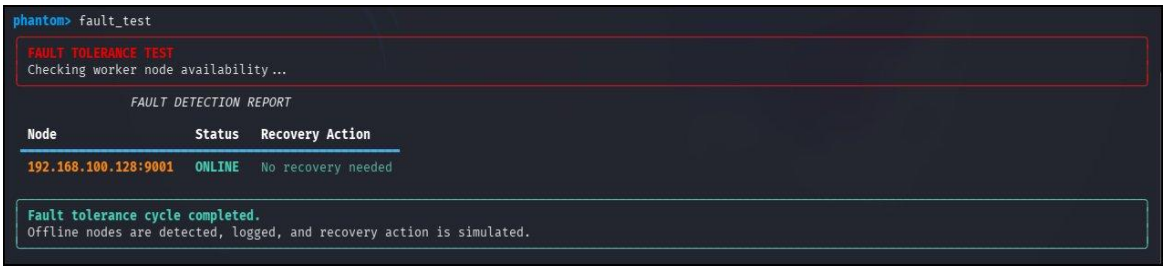


Figure 7: fault_test — Node ONLINE, no recovery action required

7.2 Scenario B — Node Offline (Recovery Triggered)

When the worker node is shut down or unreachable, the fault_test detects the failure and triggers the appropriate recovery action: tasks are marked for reassignment, and the system waits for node recovery. This simulates real-world distributed system resilience.

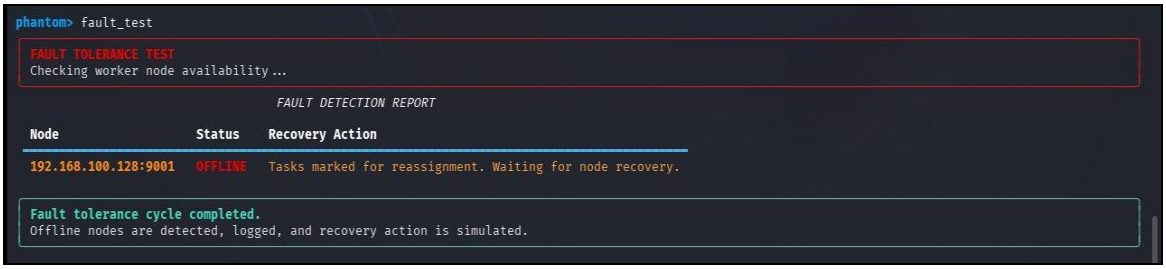


Figure 8: fault_test — Node OFFLINE detected; tasks marked for reassignment

Both scenarios demonstrate the complete fault detection lifecycle — from health-check broadcast through status classification to recovery prescription — all logged automatically to the audit system.

8. Live Dashboard

The dashboard command launches the Phantom OS Live Dashboard — a continuously refreshing ASCII panel that displays real-time CPU and RAM utilization for each connected node. This gives operators an instant visual overview of the entire distributed system without leaving the terminal.

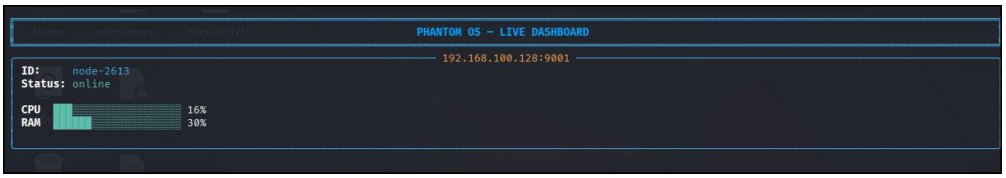


Figure 9: Live Dashboard — node-2613 at CPU 16%, RAM 30%

The dashboard polls all worker nodes at regular intervals and updates the progress bars in-place, providing a smooth, low-bandwidth monitoring experience suitable for both local and remote deployments.

9. Audit Logging — logs

Every action taken within Silent Vector is recorded to shared/phantom.log as structured JSON entries. Each log entry includes a timestamp, event type, and full details of the operation. The logs command renders the most recent audit entries directly in the shell.

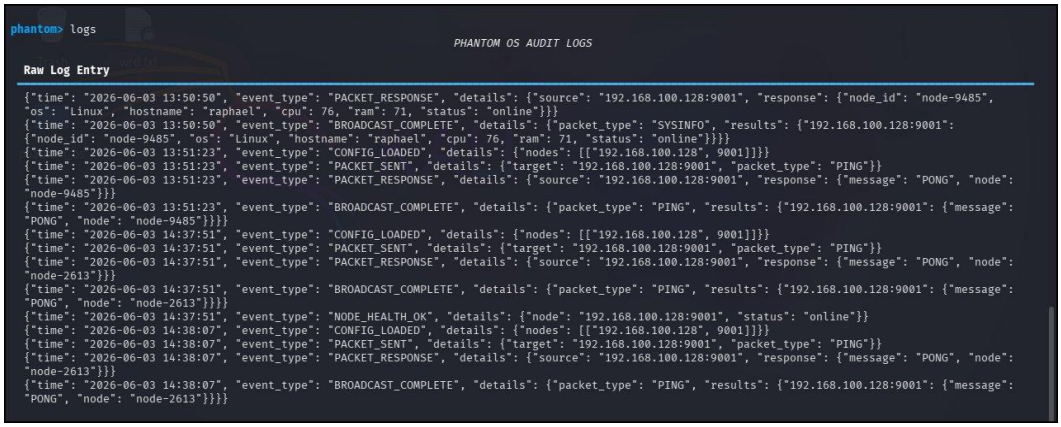


Figure 10: Audit log output — PACKET_SENT, PACKET_RESPONSE, BROADCAST_COMPLETE events

Log event types captured include:

- CONFIG_LOADED — Node configuration successfully read from config.json
- PACKET_SENT — A packet dispatched from master to a worker node
- PACKET_RESPONSE — Response received from a worker node
- BROADCAST_COMPLETE — A command successfully broadcast to all nodes
- NODE_HEALTH_OK — Node health check passed; node confirmed online

10. Cinematic Demo — auto_demo & replay

Silent Vector includes a fully scripted cinematic demonstration mode. Both auto_demo and replay execute a pre-configured sequence of distributed OS operations, making the system ideal for classroom demonstrations, presentations, and recorded walkthroughs.

10.1 Auto Demo — Multi-Step Walkthrough

The auto_demo command executes a complete end-to-end distributed OS scenario across five sequential steps:

- Step 1: Pings all nodes to confirm connectivity
- Step 2: Collects system info (CPU, RAM, OS, hostname) from all nodes
- Step 3: Starts multiple distributed tasks across nodes
- Step 4: Displays active task overview with progress bars
- Step 5: Simulates task migration to demonstrate workload balancing

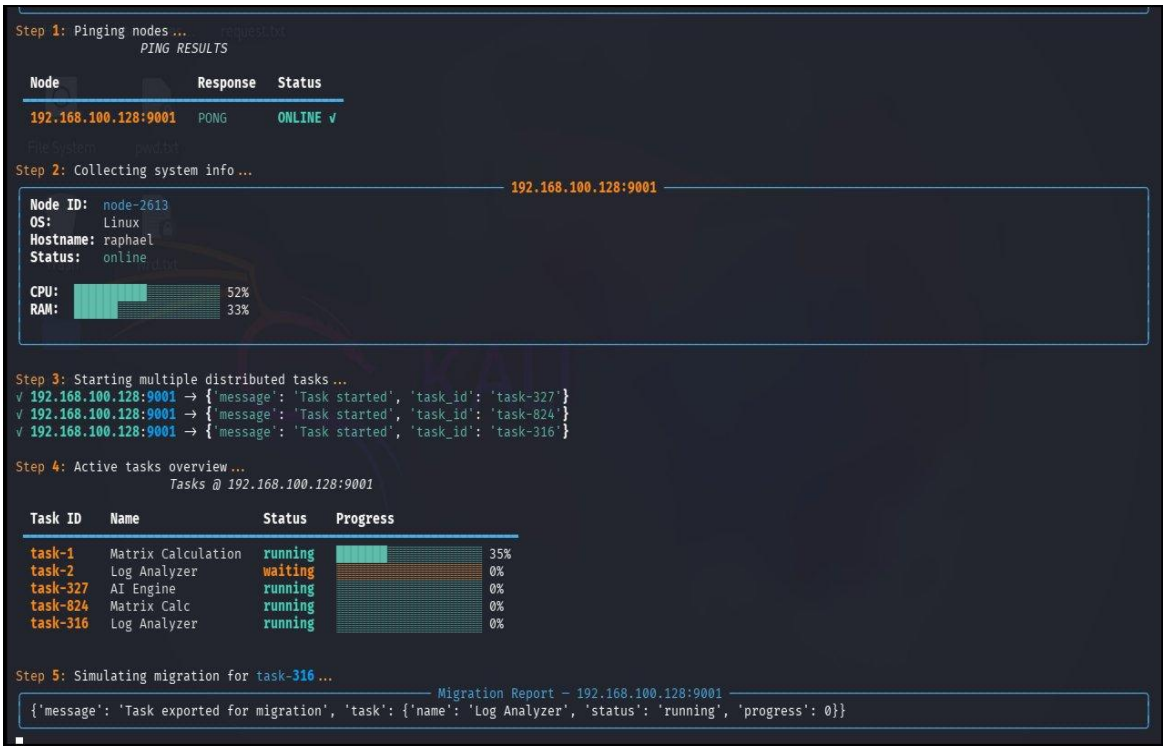


Figure 11: auto_demo — Full distributed OS scenario from ping to task migration

10.2 Replay Mode

The replay command runs a cinematic pre-scripted version of the distributed OS demo, ideal for presentations. It walks through node scanning, system info collection, task creation, and task listing in a visually polished sequence.



Figure 12: replay — Cinematic distributed OS demo with Replay-AI-Task spawned on node-2613

11. Setup & Deployment Guide

11.1 Prerequisites

- Python 3.x installed on all machines
- Network connectivity between master and worker machines
- shared/config.json configured with worker node IPs and ports

11.2 Step 1 — Start Worker Node(s)

Launch one or more worker nodes on separate terminals or remote VMs:

```
python3 worker/worker_node.py <port>
```



Figure 13: Worker node terminal showing successful startup on port 9001

11.3 Step 2 — Launch Phantom Shell

Start the master shell on the controller machine:

```
python3 master/phantom_shell.py
```

11.4 Step 3 — Configure Nodes

Edit shared/config.json to add or remove nodes from the topology:

```
{ "nodes": [["192.168.100.128", 9001]], "topology": [] }
```

12. Summary & Conclusion

Silent Vector successfully demonstrates the core principles of a distributed operating system in a safe, sandboxed Python environment. The project covers the full stack of distributed systems concepts:

Concept	Implementation in Silent Vector
Node Communication	Custom TCP packet layer with checksum validation
Task Management	Distributed task start, stop, list, and progress tracking
Task Migration	Live workload movement between nodes
Fault Tolerance	Offline detection, logging, and recovery simulation
Real-time Monitoring	Live ASCII dashboard with CPU and RAM metrics
Audit Trail	Structured JSON logging of all system events
Topology Management	Dynamic config-driven node registration
Demo / Presentation	Cinematic replay and auto_demo modes

Silent Vector — Silently watching. Always ready.